

코드 분석 리포트

app.py 완전 분석

Flask로 만든 로그인 · 회원가입 웹사이트 서버 코드 해설

login_test 프로젝트 · Python / Flask / SQLite

OVERVIEW

전체 그림 먼저 보기

웹사이트는 두 부분으로 나뉩니다.

서버 (백엔드)

`app.py`

요청을 처리하고 데이터베이스에
저장 · 조회하는 역할

화면 (프론트엔드)

`templates/*.html`

사용자 눈에 실제로 보이는 HTML 페이지

`app.py`는 "누가 `/login` 주소로 접속하면 로그인 화면을 보여주고, 로그인 버튼을 누르면 아이디 · 비밀번호를 확인해서 맞으면 들여보내준다" 같은 규칙들의 모음입니다.

준비 단계 — 도구 가져오기

```
import sqlite3
from pathlib import Path
from flask import Flask, g, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash, generate_password_hash
BASE_DIR = Path(__file__).resolve().parent
DB_PATH = BASE_DIR / "users.db"
app = Flask(__name__)
app.config["SECRET_KEY"] = "dev-secret-key-change-me"
```

import 줄들은 "이 도구들을 가져다 쓰겠다"는 선언입니다.

sqlite3: 파이썬에 기본 내장된 미니 데이터베이스 엔진. 회원 정보를 파일 하나(users.db)에 저장합니다.

Flask: 웹서버를 만드는 핵심 도구.

werkzeug.security: 비밀번호를 안전하게 암호화(해싱)해주는 도구.

BASE_DIR = app.py 파일이 있는 폴더. 폴더를 어디로 옮겨도 항상 같은 위치에 users.db를 만들 수 있습니다.

app = Flask(__name__) → 웹서버 앱을 하나 만듭니다. 이제부터 app에 규칙을 하나씩 추가해나갑니다.

SECRET_KEY → 로그인 상태(세션)를 암호화할 때 쓰는 비밀 값. 실제 서비스라면 반드시 바꿔야 합니다.

데이터베이스 연결 관리

```
def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
    return g.db
@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()
```

get_db(): 데이터베이스(users.db)에 접속하는 함수. g는 "요청 하나가 처리되는 동안만 유지되는 임시 저장소"입니다.

한 요청 안에서 get_db()를 여러 번 불러도 접속을 매번 새로 하지 않고 재사용합니다.

row_factory = sqlite3.Row → 조회 결과를 user["username"]처럼 이름으로 꺼낼 수 있게 해줍니다.

close_db(): 요청 처리가 끝나면(teardown_appcontext) 자동으로 데이터베이스 연결을 닫아줍니다.

데이터베이스(테이블) 초기 생성

```
def init_db():
    db = sqlite3.connect(DB_PATH)
    db.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            created_at TEXT DEFAULT CURRENT_TIMESTAMP
        )
    """)
    db.commit()
    db.close()
```

users라는 표(테이블)를 만드는 코드입니다. 엑셀 시트를 하나 만든다고 생각하면 쉽습니다.

id: 자동으로 1, 2, 3... 매겨지는 고유 번호

username: 아이디 (UNIQUE=중복 불가, NOT NULL=비어있으면 안 됨)

password_hash: 비밀번호를 그대로 저장하지 않고 암호화한 값

created_at: 가입한 시각 (자동 기록)

CREATE TABLE IF NOT EXISTS라서 이미 표가 있으면 다시 안 만들고 넘어갑니다.

★ 비밀번호를 그대로 저장하지 않는다는 게 중요한 보안 포인트입니다.

첫 화면(/) 규칙

```
@app.route("/")
def index():
    if session.get("user_id"):
        return redirect(url_for("dashboard"))
    return redirect(url_for("login"))
```

@app.route("/") 같은 @으로 시작하는 줄을 데코레이터라고 하는데, "이 주소(/)로 요청이 오면 바로 아래 함수를 실행해줘"라는 뜻입니다.

session은 "지금 이 사용자가 로그인했는지"를 브라우저 쿠키에 안전하게 저장해두는 곳입니다.

로그인한 사람이면 /dashboard로, 아니면 /login으로 자동 이동시킵니다.

회원가입 (/signup)

```
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        confirm = request.form.get("confirm", "")
        error = None
        if len(username) < 3:
            error = "아이디는 3자 이상이어야 합니다."
        elif len(password) < 6:
            error = "비밀번호는 6자 이상이어야 합니다."
        elif password != confirm:
            error = "비밀번호가 일치하지 않습니다."
        if error is None:
            db = get_db()
            existing = db.execute(
                "SELECT id FROM users WHERE username = ?", (username,)
            ).fetchone()
            if existing is not None:
                error = "이미 사용 중인 아이디입니다."
            else:
                db.execute(
                    "INSERT INTO users (username, password_hash) VALUES (?, ?)",
                    (username, generate_password_hash(password)),
                )
                db.commit()
            return redirect(url_for("login", signup="success"))
```

methods=["GET", "POST"] → GET(그냥 페이지를 보러 오는 것) / POST(폼을 입력하고 제출 버튼을 누른 것) 두 방식을 모두 받습니다.

검증 순서: ① 아이디 3자 미만 → ② 비밀번호 6자 미만 → ③ 비밀번호/확인 불일치 → ④ 이미 있는 아이디

generate_password_hash(password) → 비밀번호를 되돌릴 수 없는 암호문으로 바뀌어서 저장합니다.

VALUES (?, ?) 처럼 물음표를 쓰는 이유: 입력값을 SQL 문장에 직접 끼워 넣으면 SQL 인젝션 공격에 뚫릴 수 있어, 값만 안전하게 따로 전달합니다.

가입 성공 시 로그인 페이지로 이동하며 signup=success 표시를 붙여 "가입 완료" 메시지를 보여줍니다.

로그인 (/login)

```

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        db = get_db()
        user = db.execute(
            "SELECT * FROM users WHERE username = ?", (username,)
        ).fetchone()
        if user is None or not check_password_hash(user["password_hash"], password):
            return render_template(
                "login.html", error="아이디 또는 비밀번호가 올바르지 않습니다.", username=username
            )
        session.clear()
        session["user_id"] = user["id"]
        session["username"] = user["username"]
        return redirect(url_for("dashboard"))

```

입력한 아이디로 DB에서 사용자를 찾습니다.

check_password_hash(저장된 암호문, 입력한 비밀번호) → 입력한 비밀번호를 같은 방식으로 암호화해서 저장된 값과 비교합니다. 절대 복호화하지 않습니다.

아이디가 없거나 비밀번호가 틀리면 일부러 똑같은 오류 메시지를 보여줍니다. 어떤 메시지인지로 아이디 존재 여부를 알아낼 수 없게 하기 위함입니다.

맞으면 session에 user_id, username을 저장합니다. 이제부터 브라우저는 이 정보가 담긴 쿠키를 들고 다니며 "로그인함"을 증명합니다.

로그아웃 & 대시보드

```
@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))
@app.route("/dashboard")
def dashboard():
    if not session.get("user_id"):
        return redirect(url_for("login"))
    return render_template("dashboard.html", username=session.get("username"))
```

/logout → 세션(로그인 정보)을 전부 지우고 로그인 페이지로 보냅니다.

/dashboard → 보호된 페이지입니다. 세션에 user_id가 없으면(로그인 안 했으면) 강제로 로그인 페이지로 쫓아냅니다.

로그인 상태라면 환영 메시지 화면을 보여줍니다.

실행부

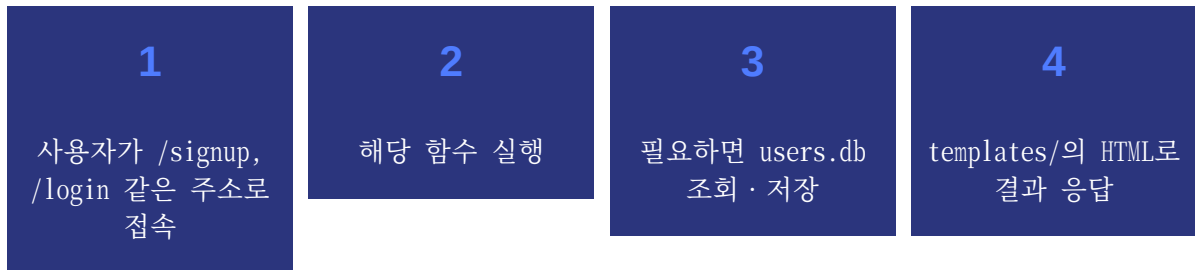
```
if __name__ == "__main__":  
    init_db()  
    app.run(debug=True)
```

python app.py로 이 파일을 직접 실행했을 때만 동작하는 코드입니다.

init_db()로 테이블이 없으면 만들고, app.run(debug=True)로 서버를 켭니다.

debug=True는 개발 중 코드 변경사항을 자동 반영하고 오류를 자세히 보여주는 옵션이라, 실제 배포 시엔 꺼야 합니다.

한 줄 요약



비밀번호는 절대 원문으로 저장하지 않고(해싱), 로그인 상태는 session(쿠키)으로 기억합니다.

핵심 보안 포인트

비밀번호 해싱 (generate_password_hash)

SQL 인젝션 방지 — 값을 직접 문자열로 끼워넣지 않고 물음표(?) 파라미터로 전달

로그인 실패 시 아이디/비밀번호를 구분하지 않는 동일한 오류 메시지